CANTINA

vault

Apex Security Review

June 22, 2026

Contents

1 Introduction **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
- 2.1 Scope . . . 3

3 Findings **4**

4 Scan #1 **4**
- 4.1 Medium Risk . . . 4
  - 4.1.1 VAUL-8 - approve_request mints claimable shares from nominal amounts after Token-2022 transfer fees are re-enabled . . . 4
  - 4.1.2 VAUL-10 - SubscriptionQueue accepts zero-asset tombstones, letting attackers lock the FIFO deposit queue at near-zero cost . . . 6
  - 4.1.3 VAUL-7 - Deposit fees can turn approved deposits into zero-share claims while still taking the full asset amount . . . 8
  - 4.1.4 VAUL-6 - Reusable request pubkeys let stale authority-signed approvals settle a different non-queued request instance . . . 10
  - 4.1.5 VAUL-16 - Large percentage-fee requests can wedge fee-enabled approvals because FeeType::get_fee overflows in u64 . . . 12
  - 4.1.6 VAUL-12 - initialize_vault blindly activates stale bootstrap state without revalidating the live mint and custody boundary . . . 14
- 4.2 Low Risk . . . 16
  - 4.2.1 VAUL-14 - SubscriptionQueue can be permanently head-blocked with deposits smaller than the fixed deposit fee . . . 16
  - 4.2.2 VAUL-1 - ConfidentialMintBurn share mints can permanently lock approved deposits and all redemptions . . . 18
  - 4.2.3 VAUL-2 - Reusable request pubkeys let stale signed queue-processing transactions settle a different request instance . . . 20

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

This scan report summarizes only the client-visible findings from Scan #1 for vault for vault. Effective severities, statuses, notes, and comments are sourced from persisted client feedback at export time.

From Jun 3, 2026 to Jun 3, 2026 Cantina conducted a security review of vault on commit hash ce2b5483de53cd015efbbdea70ecec75d976bb08. The team identified a total of **9** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Extreme Risk	0	0	0
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	6	6	0
Low Risk	3	3	0
Informational	0	0	0
Total	**9**	**9**	**0**

All included findings were marked fixed at the time of the review.

Cantina reviewed vault holistically on commit hash ce2b5483de53cd015efbbdea70ecec75d976bb08. At the time of this export, the included findings were marked fixed and no new vulnerabilities were identified.

2.1 Scope

This report covers the selected scan snapshot:

- `Scan #1`
  - `https://github.com/solana-foundation/vault` - branch `main` - commit `ce2b5483de53`

vault for `vault` was reviewed by Cantina in Scan #1. This scan-scoped export covers Scan #1 only.

This export includes 9 client-visible findings, grouped by effective severity and using persisted client statuses for report presentation.

3 Findings

4 Scan #1

Created: June 3, 2026

Repositories

- `https://github.com/solana-foundation/vault` - branch `main` - commit `ce2b5483de53`

4.1 Medium Risk

4.1.1 VAUL-8 - approve_request mints claimable shares from nominal amounts after Token-2022 transfer fees are re-enabled

✓ **Medium**

Status: Fixed

Original severity: Medium

Executive Summary The program tries to enforce a zero-transfer-fee invariant for Token-2022 asset mints, but it only checks that invariant at vault creation and deposit-request creation. It never revalidates the mint inside `approve_request`, `claim`, `cancel_request`, `reject_request`, or `withdraw_assets`. Token-2022 transfer-fee authorities can change the fee schedule after the vault already exists; the repository's own tests demonstrate that post-creation fee mutation is possible. An attacker controlling that external authority can submit a deposit request while the fee is zero, wait until the request is pending in `pending_vault`, then enable transfer fees before the vault authority approves settlement. The `pending_vault -> reserve` transfer withholds part of the transfer on the recipient side, but `approve_request` still increments `total_asset_balance` and computes claimable shares from the full nominal amount. The attacker ends up with claimable shares backed by less actual reserve custody, creating an undercollateralized vault and a later drain path against honest liquidity.

Details The zero-fee validation is present in `create_vault` and `create_deposit_request`:

```
validate_asset_mint_extensions_from_acct_info(&ctx.accounts.asset_mint.to_account_info())?;
```

But `approve_request` never repeats that check before it moves assets and books claimable value:

```
let (claimable_amount, balance_delta) = if is_deposit {
    let (deposit_fee, net_deposit) =
        get_deposit_fee_and_net(&ctx.accounts.vault.to_account_info(), original_amount)?;
    ...
    ctx.accounts.settle_deposit(seeds, net_deposit)?;
    let shares = calculate_shares(nav, decimals, net_deposit)?;
    (shares, net_deposit)
} else {
    ...
};

vault.total_asset_balance = vault
    .total_asset_balance
    .checked_add(balance_delta)?;
request.amount = claimable_amount;
request.request_state = RequestState::Claimable;
```

Token-2022 transfer-fee docs state that on every transfer, part of the amount is withheld on the recipient account, and a separate authority may change the fee configuration. The repo's own `create_deposit_request` test shows this exact post-creation mutation sequence by creating a zero-fee mint, then calling `set_transfer_fee` later.

Exploit path:

- The attacker controls the Token-2022 transfer-fee authority for the vault's asset mint.
- They create a deposit request while the transfer fee is zero, so `create_deposit_request` accepts the mint and moves the full amount into `pending_vault`.
- Before the vault authority approves the request, the attacker enables a nonzero transfer fee and waits for the new epoch to take effect.
- `approve_request` calls `settle_deposit`, which performs `transfer_checked` from `pending_vault` to `reserve` for `net_deposit`.
- The transfer credits less than `net_deposit` to `reserve`, because the fee is withheld on the recipient side.
- Despite that shortfall, `approve_request` computes shares from `net_deposit`, adds `net_deposit` to `total_asset_balance`, and marks the request `Claimable`.
- The attacker claims the full share amount and later redeems against honest reserve liquidity once fees are removed or other deposits replenish the pool.

This is not just "the user receives less because the token has fees." The vault books and tokenizes value it never actually received.

Impact Cascade

- `reserve` receives fewer assets than `approve_request` accounts for.
- `total_asset_balance` is overstated, so vault accounting diverges from custody.
- The attacker receives fully claimable shares backed by only partially funded reserve assets.
- Future honest redeemers bear the deficit when those inflated shares are redeemed.
- The same missing revalidation also leaves redeem/cancel/claim/withdraw flows exposed to post-creation fee changes.

Assumptions and Uncertainties

- The asset mint is Token-2022 and still has a live transfer-fee authority after vault creation.
- The attacker controls that authority or can trigger a fee change.
- The attacker can get a deposit request into `Pending` while fees are zero.
- The vault authority later approves requests in the normal course of operation.
- There is sufficient honest liquidity later to realize the undercollateralization.

How will the bug recipient respond? "Deposits with nonzero transfer fees are already rejected, so this path is blocked."

That response only covers request creation. The code never revalidates the mint during settlement, and the test suite itself demonstrates that transfer fees can be changed after the vault is created. The exploit specifically uses that gap between `create_deposit_request` and `approve_request`.

Why did tests miss this issue? The tests stop at verifying that `create_deposit_request` rejects a mint after `set_transfer_fee` is called. They do not cover the realistic sequence where a request is created first, the fee is changed afterward, and `approve_request` settles the already-pending request under the new fee schedule.

Recommendation Revalidate the asset mint before every asset-moving instruction and reject any nonzero active transfer fee at settlement time. Stronger still, reject Token-2022 asset mints whose transfer-fee authority is still set to an external key. If fee-bearing assets must be supported, base `request.amount` and `total_asset_balance` on observed post-transfer balance deltas or use Token-2022's fee-aware transfer flow so the credited amount is explicit.

References

- `vault/programs/async_vault/src/utils.rs#L9-L36`
- `vault/programs/async_vault/src/instructions/create_vault.rs#L94-L120`
- `vault/programs/async_vault/src/instructions/create_deposit_request.rs#L84-L91`
- `vault/programs/async_vault/src/instructions/approve_request.rs#L165-L240`
- `vault/integration-tests/src/async_vault/create_deposit_request.rs#L163-L231`
- `ai-docs/deps/solana-program/token-2022/www.solana-program.com/docs/token-2022/extensions.md#L37-L62`

4.1.2 VAUL-10 - SubscriptionQueue accepts zero-asset tombstones, letting attackers lock the FIFO deposit queue at near-zero cost

✓ **Medium**

Status: Fixed

Original severity: Medium

Executive Summary The queue-extension safety model assumes tombstones are self-limiting because canceled queued requests must stay open until `skip_canceled_queue_request` advances past them. That assumption fails for queued deposits because `create_deposit_request` never rejects `amount == 0`. A remote user can therefore create a zero-asset queued deposit, immediately cancel it, and leave behind a `Canceled` tombstone that still occupies the next FIFO slot. The cancel path decrements `pending_async_requests` immediately, so the attacker can repeat this cycle indefinitely without ever tying up underlying assets. Later, when the queue reaches those tombstones, `skip_canceled_queue_request` closes each request and returns its rent to the attacker, so the net attack cost is only transaction fees plus temporary rent float. Every honest queued deposit behind the tombstones is blocked until the backlog is skipped one request at a time. This turns the rollback path itself into a practical system-wide queue-lock primitive.

Details `create_deposit_request` allows zero-amount requests, still transfers `0`, writes a live `Pending` request, increments the global pending counter, and assigns a queue ID when the subscription queue extension is present:

```
pub fn handler(ctx: Context<CreateDepositRequest>, args: RequestArgs) -> Result<()> {
    ctx.accounts.vault.assert_unpaused_and_initialized()?;
    ...
    ctx.accounts
        .transfer_assets_from_user_to_pending_vault(args.amount)?;
    ...
    ctx.accounts.request.set_inner(Request {
        request_type: RequestType::Deposit,
        request_state: RequestState::Pending,
        amount: args.amount,
        ...
    });
    ctx.accounts.vault.pending_async_requests = ctx
        .accounts
        .vault
        .pending_async_requests
        .checked_add(1)
        .ok_or(AsyncVaultError::ArithmeticError)?;
    if let Some(id) = next_queue_request_id::<SubscriptionQueue>(&ctx.accounts.vault.to_account_info
    ())? {
        init_request_extension(&ctx.accounts.request.to_account_info(), &SubscriptionQueueRequest {
    id })?;
    }
    Ok(())
}
```

`cancel_queued_deposit_request` then refunds the same amount, marks the request `Canceled`, and decrements `pending_async_requests` immediately, but deliberately keeps the account alive as a tombstone:

```
pub fn handler(ctx: Context<CancelQueuedDepositRequest>) -> Result<()> {
    ctx.accounts.vault.assert_unpaused_and_initialized()?;
    require!(ctx.accounts.request.request_state == RequestState::Pending, ...);
    ctx.accounts.validate_has_subscription_queue_extension()?;

    let refund_amount = ctx.accounts.request.amount;
    ctx.accounts.transfer_assets_to_user(refund_amount)?;

    ctx.accounts.request.request_state = RequestState::Canceled;
    ctx.accounts.vault.pending_async_requests = ctx
        .accounts
        .vault
        .pending_async_requests
        .checked_sub(1)
        .ok_or(AsyncVaultError::ArithmeticError)?;
    Ok(())
}
```

The tombstone is later removed by `skip_canceled_queue_request`, which closes the request and returns

rent to the original owner:

```
#[account(
    mut,
    close = owner,
    constraint = request.request_state == RequestState::Canceled @ ...,
    has_one = vault,
    has_one = owner,
)]
pub request: Account<'info, Request>;
```

The attack sequence is:

- Wait for or create a vault with `SubscriptionQueue` enabled and no minimum-subscription extension (or a zero minimum).
- Submit a queued deposit request with `amount = 0`.
- Call `cancel_queued_deposit_request`. No assets are ever locked, but the request remains on-chain as the next tombstone and `pending_async_requests` is restored.
- Repeat steps 2-3 arbitrarily many times. Each iteration adds another tombstone without consuming asset capital.
- Every later queued deposit is now blocked behind attacker tombstones because FIFO approval/rejection requires strict ID order. Recovery requires one `skip_canceled_queue_request` transaction per tombstone.
- As the backlog is eventually skipped, the attacker receives the request-account rent back, so the attacker's permanent cost is only transaction fees.

This violates the queue document's central anti-spam claim that rent alone makes tombstone abuse expensive. The rollback path returns the user's value before the request leaves the queue, but the queue slot remains live and globally blocking.

Impact Cascade

- A single remote user can manufacture an arbitrarily long backlog of queued deposit tombstones without locking underlying assets.
- Every honest queued deposit behind that backlog becomes unapprovable and unrejectable until the tombstones are skipped one-by-one.
- Because rent is returned on skip, the attack is economically close to fee-only griefing instead of being self-limited by capital.
- Operators lose the ability to keep the deposit queue live without continuous janitorial transactions.
- The FIFO deposit extension can be turned into a practical long-lived queue lock for all later depositors.

Assumptions and Uncertainties

- The target vault has `SubscriptionQueue` enabled.
- The target vault does not also enforce a positive `MinSubscription` threshold; this extension is optional and disabled by default.
- SPL/Token-2022 zero-amount transfers continue to succeed; the repository's own integration test already treats a zero-amount deposit request as valid.

How will the bug recipient respond? "Tombstones already cost rent, and deployers who care can configure `MinSubscription`."

That response does not resolve the issue. The queue documentation explicitly presents rent as the anti-spam control for tombstones, but `skip_canceled_queue_request` returns that rent to the attacker. Optional deployment-time extensions do not change the fact that the core queue rollback path accepts zero-asset requests and converts them into globally blocking tombstones.

Why did tests miss this issue? The test suite explicitly normalizes zero-amount deposit requests on the non-queued path, but the queue tests only exercise positive-value queued deposits. There is no integration test that combines `amount = 0` with `SubscriptionQueue`, queued cancellation, and repeated tombstone creation.

Recommendation The primary fix is to make queued deposits economically non-zero in core logic rather than relying on optional extensions.

```
require!(args.amount > 0, AsyncVaultError::InsufficientDepositAmount);
```

Defense-in-depth improvements:

- Reject queued cancellations for zero-amount requests even if legacy zero requests exist.
- Keep a separate counter for live tombstones or include canceled queued requests in a bounded queue-length metric until `skip_canceled_queue_request` actually closes them.
- Consider batching or auto-advancing consecutive tombstones to avoid one-transaction-per-tombstone recovery.

References

- `vault/programs/async_vault/src/instructions/create_deposit_request.rs#L84-L131`
- `vault/programs/async_vault/src/extensions/subscription_queue/instructions/cancel_queued_-deposit_request.rs#L93-L112`
- `vault/programs/async_vault/src/extensions/fifo_queues/skip_canceled_queue_request.rs#L12-L30`
- `vault/programs/async_vault/docs/extensions/SubscriptionQueue.md#L71-L102`
- `vault/integration-tests/src/async_vault/cancel_request.rs#L15-L18`
- `vault/integration-tests/src/async_vault/cancel_request.rs#L62-L77`

> **Client Feedback**
> Added check for amount > 0

4.1.3 VAUL-7 - Deposit fees can turn approved deposits into zero-share claims while still taking the full asset amount

✓ **Medium**

Status: Fixed

Original severity: Medium

Executive Summary `create_deposit_request` moves the user's assets into `pending_vault` immediately, and `approve_request` later applies any configured deposit fee before converting the remaining assets into shares. The bug is that the approval path never rejects `net_deposit == 0` or `shares == 0`. After fee transfer and reserve settlement, the program marks the request `Claimable`, overwrites `request.amount` with `0`, and `claim` mints exactly `request.amount` shares. The depositor therefore loses the deposited assets permanently while receiving no shares at all.

This is not just a cosmetic dust edge. Percentage fees round up, so even ordinary low-bps fees can confiscate the smallest deposits entirely, and fixed fees can zero out much larger deposits. Because approval closes the cancellation window, the user cannot recover after the authority settles the request. The result is direct user fund loss plus accounting mutation in favor of the fee recipient and/or existing shareholders.

Details The deposit flow transfers user assets into escrow before any later share-conversion safety check exists:

```
pub fn transfer_assets_from_user_to_pending_vault(&self, amount: u64) -> Result<()> {
    let cpi_ctx = CpiContext::new(
        self.asset_token_program.key(),
        TransferChecked {
            from: self.user_token_account.to_account_info(),
            mint: self.asset_mint.to_account_info(),
            to: self.pending_vault.to_account_info(),
            authority: self.user.to_account_info(),
        },
    );
    token_interface::transfer_checked(cpi_ctx, amount, self.asset_mint.decimals)
}
```

During approval, the program computes the deposit fee, transfers that fee out, settles the remaining assets into reserve, and only then derives shares with a floored conversion:

```
let (deposit_fee, net_deposit) =
    get_deposit_fee_and_net(&ctx.accounts.vault.to_account_info(), original_amount)?;
if deposit_fee > 0 {
    ctx.accounts.transfer_asset_from_pending_vault(
        fee_recipient_token_account_info.to_account_info(),
        deposit_fee,
        seeds,
    )?;
}
ctx.accounts.settle_deposit(seeds, net_deposit)?;
```

```
let shares = calculate_shares(nav, decimals, net_deposit)?;
```

`get_deposit_fee_and_net` only enforces `amount >= fee`; it allows `net_deposit == 0`:

```
let fee = get_deposit_fee(vault_info, amount)?;
let net = amount.checked_sub(fee).ok_or(AsyncVaultError::ArithmeticError)?;
```

`calculate_shares` floors and happily returns `0` for a positive-but-too-small post-fee deposit:

```
let shares = u128::from(net_amount)
    .checked_mul(precision)?
    .checked_div(nav)?;
Ok(u64::try_from(shares)?)
```

The request is then finalized with that zero output:

```
vault.total_asset_balance = vault.total_asset_balance.checked_add(balance_delta)?;
request.amount = claimable_amount;
request.price = nav;
request.request_state = RequestState::Claimable;
```

Finally, `claim` mints exactly `request.amount`, so a zero-share approval closes out with zero shares minted:

```
let amount = request.amount;
match request.request_type {
    RequestType::Deposit => ctx.accounts.deposit(seeds, amount)?,
    RequestType::Redeem => ctx.accounts.redeem(seeds, amount)?,
}
```

Concrete exploitable conditions:

- A deposit fee extension is enabled.
- The user submits a deposit where the post-fee amount is zero, or the post-fee amount is still positive but `floor(net_deposit * 10^decimals / nav) == 0`.
- The authority approves the request.
- The user irreversibly loses the deposit while `claim` produces no shares.

This is especially easy to trigger with rounded-up percentage fees. For example, `FeeType::Percentage` computes `ceil(total_amount * bps / 10_000)`, so even a nominal low-bps fee can consume a one-unit deposit entirely.

Impact Cascade

- Depositors can lose the full approved deposit while receiving zero shares.
- The request becomes `Claimable`, so the owner loses the ability to cancel and recover.
- Deposit fees can route the entire deposit to the fee recipient, or small positive post-fee deposits can enrich existing shareholders without minting any claim to the depositor.
- Vault accounting mutates as if settlement succeeded, which hides the confiscation behind a normal-looking approval/claim lifecycle.

Assumptions and Uncertainties

- The vault has a deposit fee configured, or the post-fee deposit can still floor to zero shares at the current NAV.
- The authority approves the request instead of rejecting it.
- The user submits a small enough deposit for the zero-output path to trigger.

How will the bug recipient respond? "Users should avoid tiny deposits, and integrators can configure minimum subscriptions if they care about dust."

That does not rebut the bug. The program already fails closed when a redeem converts to zero gross assets, proving the designers understand zero-output settlement is unsafe. Here, the live deposit approval path still finalizes a positive-value deposit into a zero-share claim and permanently removes the user's recovery path.

Why did tests miss this issue? The fee test suite only covers comfortably-sized deposits that still mint positive shares after fees, and the math unit tests only assert that `calculate_shares` returns `0` for zero input. There is no test for a positive approved deposit whose post-fee conversion floors to zero shares.

Recommendation Fail closed before moving any fee or reserve assets when the claimable share output would be zero.

```
let (deposit_fee, net_deposit) = get_deposit_fee_and_net(&ctx.accounts.vault.to_account_info(),
    original_amount)?;
require!(net_deposit > 0, AsyncVaultError::ArithmeticError);
let shares = calculate_shares(nav, decimals, net_deposit)?;
require!(shares > 0, AsyncVaultError::ArithmeticError);
```

Also consider enforcing the same invariant earlier at request creation through a preview/minimum-deposit check so users never enter an approval path that can settle to zero shares.

References

- vault/programs/async_vault/src/instructions/create_deposit_request.rs#L68-L120
- vault/programs/async_vault/src/instructions/approve_request.rs#L166-L240
- vault/programs/async_vault/src/extensions/fee/processor.rs#L127-L140
- vault/programs/async_vault/src/utils.rs#L57-L69
- vault/programs/async_vault/src/instructions/claim.rs#L149-L160
- vault/libs/vault_common/src/fee.rs#L25-L39
- vault/integration-tests/src/async_vault/extenstions/fees.rs#L143-L180

4.1.4 VAUL-6 - Reusable request pubkeys let stale authority-signed approvals settle a different non-queued request instance

✓ **Medium**

Status: Fixed

Original severity: Medium

Executive Summary Non-queued requests are identified only by the mutable `request` account address. The program lets users create requests at arbitrary signer-controlled addresses and then fully closes those accounts on `claim`, `cancel_request`, and `reject_request`. After close, the same keypair can be reused to create a fresh request at the same pubkey. `approve_request` does not bind the signed approval to any immutable request instance data such as the original amount, owner, request type, or creation slot; it reads whatever `Request` state exists at execution time. A malicious transaction composer can therefore obtain an authority signature for a harmless request, withhold broadcast, wait for that request pubkey to be closed and recreated as a larger redeem request in the same vault, and then replay the stale approval. The stale signature settles the new request and can move reserve assets into the shared `pending_vault` without any fresh authority intent.

Details Request accounts are created with plain `init` and no PDA seeds in both deposit and redeem creation, so the address is entirely chosen by the user-provided request keypair:

```
#[account(
    init,
    space = 8 + Request::INIT_SPACE + compute_request_extension_space(&vault.to_account_info()),
    payer = user,
)]
pub request: Account<'info, Request>,
```

The same request account is fully closed by ordinary lifecycle instructions:

```
#[account(
    mut,
    close = owner,
    has_one = owner @ AsyncVaultError::InvalidRequest,
    has_one = vault @ AsyncVaultError::InvalidRequest,
)]
pub request: Box<Account<'info, Request>>,
```

and similarly in `cancel_request` / `reject_request`:

```
#[account(
    mut,
    close = user,
    constraint = request.owner == user.key() @ AsyncVaultError::UnauthorizedSigner,
    has_one = vault,
)]
```

```
pub request: Box<Account<'info, Request>>,
```

Once the original request is closed, the same keypair can fund a new `Request` at the same address. `approve_request` only checks the live account contents at execution time:

```
#[account(
    mut,
    has_one = vault @ AsyncVaultError::InvalidRequest,
)]
pub request: Box<Account<'info, Request>>,

require!(
    matches!(ctx.accounts.request.request_state, RequestState::Pending),
    AsyncVaultError::RequestNotPending
);

let is_deposit = matches!(ctx.accounts.request.request_type, RequestType::Deposit);
let original_amount = ctx.accounts.request.amount;
```

There is no commitment to the originally reviewed request instance beyond the reusable pubkey. In a non-queued vault, an attacker can:

- Create a small benign request at pubkey `R` and induce the authority to sign `approve_request` for `R`.
- Withhold the signed transaction.
- Close `R` through a normal path (`claim`, `cancel_request`, or `reject_request`).
- Recreate `R` as a much larger redeem request in the same vault.
- Submit the stale authority-signed approval.

Because `approve_request` re-reads `request_type`, `amount`, and `request_state` from the new account instance, the stale signature approves the attacker-chosen replacement request and settles the new redeem from reserve custody.

Impact Cascade

- A stale approval for a harmless request can be replayed onto a larger replacement redeem request.
- Reserve assets move into `pending_vault` for the attacker-chosen replacement request without fresh authority review.
- The replacement request becomes claimable and can be claimed through the normal redeem flow.
- The vault loses the guarantee that an authority signature corresponds to the reviewed request instance.

Assumptions and Uncertainties

- The attacker can obtain an authority signature before broadcast, for example through a malicious transaction composer, relay, or signing workflow that separates signing from submission.
- The vault is not using the queue extension path already covered by the existing queued-request replay finding.
- The replacement request is created in the same vault so the signed account metas remain valid.
- The attacker controls the original request keypair and can therefore reuse the same pubkey after close.
- The authority does not independently bind approvals to off-chain request metadata before signing.

How will the bug recipient respond? "The authority signed the exact request address, so this is not a replay against a different request."

That response misses the core issue: the request *address* is reused for a fresh `Request` instance after close. The signature is not bound to immutable instance data such as the original amount, type, owner, or creation slot, so the approval can authorize a different request object that happens to live at the same pubkey.

Why did tests miss this issue? The current suite exercises request reuse only in the queued lane that already has its own finding. The non-queued `approve_request` tests never close a request, recreate a new one at the same pubkey, and replay a previously signed authority approval against the replacement instance.

Recommendation Bind authority approvals to immutable request-instance data instead of the bare pubkey alone. Practical options include:

- make request accounts PDAs derived from `(vault, owner, monotonically increasing nonce)` so a closed request address cannot be reused;
- store a unique request nonce / sequence in the account and require the approval instruction to carry and verify it;
- include and verify immutable fields such as `created_at`, `request_type`, `amount`, and `owner` in any off-chain approval payload.

A PDA-based instance identifier is the cleanest fix because it removes address reuse at the source.

References

- vault/programs/async_vault/src/instructions/create_deposit_request.rs#L41-L46
- vault/programs/async_vault/src/instructions/create_redeem_request.rs#L38-L43
- vault/programs/async_vault/src/instructions/claim.rs#L35-L42
- vault/programs/async_vault/src/instructions/cancel_request.rs#L35-L41
- vault/programs/async_vault/src/instructions/reject_request.rs#L35-L40
- vault/programs/async_vault/src/instructions/approve_request.rs#L33-L37
- vault/programs/async_vault/src/instructions/approve_request.rs#L165-L240
- vault/programs/async_vault/src/instructions/approve_request.rs#L132-L138

> **Client Feedback**
> Durable nonce related, will have expected fields in instruction data rather than change the whole architecture.

4.1.5 VAUL-16 - Large percentage-fee requests can wedge fee-enabled approvals because FeeType::get_fee overflows in u64

✓ **Medium**

Status: Fixed

Original severity: Medium

Executive Summary `vault_common::FeeType::get_fee` computes percentage fees by multiplying `total_amount * bps` in `u64`. That arithmetic overflows far below the actual `u64` token limit even though the final rounded fee would still fit comfortably in `u64`. `approve_request` calls this helper for both deposit and redeem approvals, so a live vault with any nonzero percentage fee can accept a request that later becomes mathematically unapprovable. On FIFO-enabled vaults, a first-in-line request of this size blocks every later request of the same queue until the authority notices the arithmetic failure and sends a separate reject transaction. This turns a normal user request into a queue-wide settlement lock that bypasses the intended approval path.

Details The percentage-fee path multiplies in `u64`:

```
pub fn get_fee(self, total_amount: u64) -> Result<u64, VaultMathError> {
    match self {
        FeeType::Percentage { bps } => {
            let fee = total_amount
                .checked_mul(bps.into())
                .ok_or(VaultMathError::ArithmeticError)?
                .checked_add(9_999)
                .ok_or(VaultMathError::ArithmeticError)?
                .checked_div(10_000)
                .ok_or(VaultMathError::ArithmeticError)?;
            Ok(fee)
        }
        FeeType::FixedAmount { amount } => Ok(amount),
    }
}
```

`approve_request` routes every fee-bearing approval through that helper:

```
let (deposit_fee, net_deposit) =
    get_deposit_fee_and_net(&ctx.accounts.vault.to_account_info(), original_amount)?;
...
let (withdraw_fee, net_assets) =
    get_withdrawal_fee_and_net(&ctx.accounts.vault.to_account_info(), assets)?;
```

Because the multiplication is done before the division, the request fails once:

`total_amount > floor((u64::MAX - 9_999) / bps)`.

Example thresholds:

- `10_000` bps (100%): overflow above `1,844,674,407,370,954` base units.
- `5,000` bps (50%): overflow above `3,689,348,814,741,908` base units.
- `1,000` bps (10%): overflow above `18,446,744,073,709,541` base units.

At 9 decimals, the 100% threshold is only about `1,844,674.407370954` whole tokens; at 50%, about `3,689,348.814741908` whole tokens. Those sizes are realistic for RWA and stablecoin-style vaults.

Exploit path:

- A vault enables any nonzero percentage deposit or withdrawal fee.
- The attacker creates a deposit or redeem request above the overflow threshold for that `bps`.
- The request is accepted and, if the FIFO queue extension is active, takes the next queue slot.
- Every `approve_request` attempt on that request reverts inside `FeeType::get_fee` with `ArithmeticError` before settlement.
- Later queued requests cannot be approved until the authority sends a separate reject transaction (or the attacker voluntarily cancels, which they need not do).

This is not a purely theoretical max-`u64` edge: the failure boundary falls into ordinary whole-token ranges whenever the configured fee is large.

Impact Cascade

- A single oversized request becomes permanently unapprovable through the normal fee path.
- On `SubscriptionQueue` or `RedemptionQueue`, one attacker-controlled head request blocks all later approvals in that queue.
- Users behind the attacker remain stuck until the authority performs manual cleanup.
- The lockup is triggered with a request shape the program explicitly accepts at creation time.

Assumptions and Uncertainties

- The vault has a nonzero percentage deposit fee or withdrawal fee.
- The attacker can create a request above the overflow threshold for that `bps`.
- Queue-wide impact requires the corresponding FIFO queue extension to be enabled.
- The authority can recover by rejecting the request, so this is a long-lived but recoverable lock rather than an irrecoverable one.

How will the bug recipient respond? "The authority can just reject the bad request, so this is only an operational nuisance."

The problem is that the request passes every creation-time check and becomes impossible to approve only after it reaches the live settlement path. On FIFO vaults that means one attacker request can halt all later approvals until a privileged actor intervenes with an explicit cleanup transaction, which is exactly the kind of long-lived queue lock the extensions are meant to avoid.

Why did tests miss this issue? The fee tests cover ordinary fixed-fee and low-bps percentage examples only. There is no test near the overflow boundary, and no queue test combines fee-enabled approvals with high-value requests.

Recommendation Move percentage fee arithmetic to `u128` and convert back to `u64` only after the final division, mirroring the safer arithmetic already used in `get_withdraw_fee_when_redeeming` and `get_deposit_fee_when_minting`.

```
let fee = u128::from(total_amount)
    .checked_mul(u128::from(bps))?
    .checked_add(9_999)?
    .checked_div(10_000)?;
let fee = u64::try_from(fee)?;
```

As a defense in depth measure, reject requests whose amount would overflow the current fee configuration at request-creation time instead of discovering the failure only during approval.

References

- vault/libs/vault_common/src/fee.rs#L25-L41
- vault/programs/async_vault/src/extensions/fee/processor.rs#L99-L140
- vault/programs/async_vault/src/instructions/approve_request.rs#L166-L213
- vault/programs/async_vault/src/extensions/fifo_queues/fifo_queue.rs#L69-L96

4.1.6 VAUL-12 - initialize_vault blindly activates stale bootstrap state without revalidating the live mint and custody boundary

✓ **Medium**

Status: Fixed

Original severity: Medium

Executive Summary `initialize_vault` is supposed to be the final step that makes a vault live, yet it performs no final verification of the share mint, asset mint, reserve account, or pending-vault account. The instruction takes only the `vault` account plus an authority signature and then flips `initialized = true`. As a result, the live-transition step does not atomically prove that the share mint is still controlled by the vault PDA, that the asset mint still satisfies the vault's compatibility assumptions, or that the authority is activating the same mint/custody state that existed when `create_vault` ran. Any mutation to the off-program mint policy surface between `create_vault` and `initialize_vault` is therefore invisible to the activation ceremony. This is a distinct bootstrap failure even when the correct authority created the vault originally: the program never gives that authority an on-chain checkpoint to bind activation to reviewed mint and custody state.

Details The entire activation path is a single boolean write:

```
#[derive(Accounts)]
pub struct InitializeVault<'info> {
    pub authority: Signer<'info>,

    #[account(
        mut,
        constraint = authority.key() == vault.authority @ AsyncVaultError::UnauthorizedSigner,
    )]
    pub vault: Account<'info, Vault>,
}

pub fn handler(ctx: Context<InitializeVault>) -> Result<()> {
    ctx.accounts.vault.assert_uninitialized()?;
    ctx.accounts.vault.initialized = true;
    Ok(())
}
```

By contrast, `create_vault` stores only pubkeys and assumes the trust boundary created at that moment will remain valid forever:

```
ctx.accounts.vault.set_inner(Vault {
    asset_mint: ctx.accounts.asset_mint.key(),
    share_mint: ctx.accounts.share_mint.key(),
    vault_token_account: ctx.accounts.reserve.key(),
    pending_vault: ctx.accounts.pending_vault.key(),
    authority: args.authority,
    ...
});
```

There is no activation-time check that:

- the live share mint still has `mint_authority == vault`,
- the live share mint still belongs to the same token-program domain reviewed at bootstrap,
- the live asset mint still satisfies the vault's compatibility assumptions,
- the reserve and pending vault still represent the custody state the authority intended to activate.

Exploit path:

- A vault is created successfully.
- Before `initialize_vault`, some external actor who still controls mutable mint-side policy surfaces changes the effective mint/custody environment the vault will rely on once live.
- The authority calls `initialize_vault`, expecting an on-chain activation checkpoint.
- The program blindly flips `initialized = true` without loading any of the live mint/custody accounts.
- All later privileged flows (`approve_request`, `claim`, `withdraw_assets`, cancel/reject paths) now trust a bootstrap state that was never revalidated at activation.

The root cause is that the protocol treats initialization as a final trust-establishment step, but the instruction never actually checks the trust assumptions it is finalizing.

Impact Cascade

- The authority cannot atomically activate only the mint/custody state they actually reviewed.
- Mutable off-program mint policy can change after `create_vault` yet before activation, and the live transition will not detect it.
- The activation ceremony becomes a blind endorsement of stale bootstrap data rather than a final safety checkpoint.
- Downstream minting, burning, settlement, and withdrawal logic operate on assumptions that may already be invalid when the vault goes live.

Assumptions and Uncertainties

- Some portion of the bootstrap state remains mutable outside the vault program between `create_vault` and `initialize_vault`.
- The vault authority relies on `initialize_vault` as the final on-chain moment to approve that state.
- Later vault flows trust the stored `Vault` pubkeys plus the live mint accounts they point to.

How will the bug recipient respond? "The authority can inspect the mints before initializing, so an explicit revalidation instruction is unnecessary."

The inspection is not atomic because `initialize_vault` does not take the reviewed mint/custody accounts as inputs. Even a careful authority cannot tie their initialization signature to the exact state they inspected in the same transaction. A bootstrap trust boundary that depends on off-chain review rather than enforced on-chain revalidation is not actually frozen by initialization.

Why did tests miss this issue? The initialization tests only prove that the right signer can toggle `initialized` and that the stored fields are preserved byte-for-byte. They never mutate bootstrap-adjacent mint or custody state between `create_vault` and `initialize_vault`, and they never require `initialize_vault` to re-supply any live mint/custody accounts for validation.

Recommendation Turn `initialize_vault` into a real finalization check.

Require the instruction to take the live `share_mint`, `asset_mint`, `reserve`, and `pending_vault` accounts, then verify at minimum:

- `share_mint.mint_authority == Some(vault.key())`,
- the share mint still belongs to the expected token program,
- the asset mint still passes compatibility validation,
- `reserve` and `pending_vault` still match the stored pubkeys and token-program assumptions.

Only after those checks should `initialized` be set to `true`.

References

- `vault/programs/async_vault/src/instructions/create_vault.rs#L94-L123`
- `vault/programs/async_vault/src/instructions/initialize_vault.rs#L5-L21`
- `vault/programs/async_vault/src/state/async_vault.rs#L7-L36`
- `vault/integration-tests/src/async_vault/initialize_vault.rs#L12-L105`

4.2 Low Risk

4.2.1 VAUL-14 - SubscriptionQueue can be permanently head-blocked with deposits smaller than the fixed deposit fee

✓ **Low**

Status: Fixed

Original severity: Low

Executive Summary A vault that enables both `SubscriptionQueue` and a fixed `DepositFee` accepts queued deposits that are too small to survive fee deduction. The request still receives the next FIFO queue ID at creation time, but `approve_request` later computes `net_deposit = amount - fee` and aborts with `ArithmeticError` when the fixed fee exceeds the deposited amount. Because `SubscriptionQueue` enforces strict in-order processing, that unapprovable request becomes the mandatory head of the queue and every later deposit is blocked behind it until the authority explicitly rejects it. An unprivileged user can therefore keep inserting sub-fee deposits to repeatedly force manual head cleanup and stall the live deposit queue for all later users. This is not an admin-misconfiguration-only edge case: any positive fixed deposit fee creates a whole range of otherwise-valid deposit sizes that the public create path still accepts.

Details `create_deposit_request` accepts any deposit amount that passes the optional minimum-subscription check, transfers the assets into `pending_vault`, and assigns a queue ID whenever `SubscriptionQueue` is active:

```
extensions::min_subscription::check_min_subscription_amount(
    &ctx.accounts.vault.to_account_info(),
    args.amount,
)?;
ctx.accounts
    .transfer_assets_from_user_to_pending_vault(args.amount)?;
...
if let Some(id) =
    next_queue_request_id::<SubscriptionQueue>(&ctx.accounts.vault.to_account_info())?
{
    init_request_extension(
        &ctx.accounts.request.to_account_info(),
        &SubscriptionQueueRequest { id },
    )?;
}
```

The fixed-fee TLV stores an arbitrary constant `amount` and the approval path applies that fee only later:

```
pub fn get_fee(self, total_amount: u64) -> Result<u64, VaultMathError> {
    match self {
        FeeType::Percentage { bps } => { ... }
        FeeType::FixedAmount { amount } => Ok(amount),
    }
}
```

During approval, the queue check runs first, making the request the mandatory next item, and only then does the program subtract the fee:

```
if is_deposit {
    check_and_advance_queue::<SubscriptionQueue, SubscriptionQueueRequest>(
        &ctx.accounts.vault.to_account_info(),
        &ctx.accounts.request.to_account_info(),
    )?;
}
...
let (deposit_fee, net_deposit) =
    get_deposit_fee_and_net(&ctx.accounts.vault.to_account_info(), original_amount)?;
```

`get_deposit_fee_and_net` fails closed on `amount < fee`:

```
let fee = get_deposit_fee(vault_info, amount)?;
let net = amount
    .checked_sub(fee)
    .ok_or(AsyncVaultError::ArithmeticError)?;
```

That makes the exploit straightforward:

- Find a live vault with `SubscriptionQueue` enabled and a fixed positive `DepositFee`.
- Submit a deposit request with `0 < amount < fixed_fee`.
- The request is accepted, escrowed, and assigned the next queue ID.
- Because FIFO is strict, no later queued deposit can be approved out of order.
- When the authority tries to approve the head request, approval reverts on `amount.checked_sub(fee)`.
- The only escape hatch is a manual `reject_request`, so the attacker can keep repeating the pattern and continuously stall queued deposits.

Impact Cascade

- Any unprivileged user can create a queue-head deposit that is valid at creation time but impossible to approve.
- Strict FIFO turns that single request into a global blocker for every later queued deposit.
- The authority must manually reject each poison-pill request before normal deposit processing can resume.
- An attacker can repeat the pattern indefinitely, degrading the queue into an operator-driven janitorial process instead of an automated fairness mechanism.

Assumptions and Uncertainties

- The target vault has both `SubscriptionQueue` and a fixed `DepositFee` enabled.
- The target vault either has no `MinSubscription` extension or sets it below the fixed fee range the attacker uses.
- The authority processes requests via the normal queue path and cannot bypass the FIFO head.

How will the bug recipient respond? "The authority can just reject those requests, so this is only a nuisance."

That misses the queue-level impact. The bug is that the public create path accepts a request the queue settlement path can never approve, and `SubscriptionQueue` makes that malformed head request block every later deposit until the authority spends an extra reject transaction clearing it. Repeating the attack keeps the live deposit queue stalled.

Why did tests miss this issue? The fee-extension tests only approve comfortably sized deposits where `deposit_amount` is far larger than the fixed fee, and the queue tests only use ordinary positive deposits. There is no integration test that combines `SubscriptionQueue` with a fixed deposit fee and then submits a deposit below that fee amount.

Recommendation Reject unapprovable deposits at creation time.

A robust fix is to read the active deposit-fee extension inside `create_deposit_request` and reject any request whose post-fee amount would be zero or negative before assigning a queue ID. As defense in depth, add an explicit approval-time error such as `DepositAmountBelowNetMinimum` and cover the queue + fixed-fee composition in integration tests.

References

- `vault/programs/async_vault/src/instructions/create_deposit_request.rs#L84-L133`
- `vault/libs/vault_common/src/fee.rs#L25-L40`
- `vault/programs/async_vault/src/extensions/fee/processor.rs#L127-L133`
- `vault/programs/async_vault/src/instructions/approve_request.rs#L147-L188`
- `vault/programs/async_vault/docs/extensions/SubscriptionQueue.md#L29-L37`
- `vault/programs/async_vault/docs/extensions/SubscriptionQueue.md#L132-L137`
- `vault/integration-tests/src/async_vault/extenstions/fees.rs#L143-L208`

4.2.2 VAUL-1 - ConfidentialMintBurn share mints can permanently lock approved deposits and all redemptions

✓ **Low**

Status: Fixed

Original severity: Low

Executive Summary `create_vault` accepts any Token-2022 share mint as long as its visible supply is zero. It never rejects the `ConfidentialMintBurn` extension or checks whether the vault's later share lifecycle is compatible with that mint. That is dangerous because the async-vault program always uses the *standard public* `mint_to` and `burn` instructions for shares: deposits are paid out in `claim` via `mint_to`, redemptions start in `create_redeem_request` via `burn`, and redeem rollbacks in `cancel_request` / `reject_request` also use `mint_to`.

The Token-2022 processor forbids those regular public mint/burn paths when `ConfidentialMintBurn` is present. A vault bootstrapped around such a share mint therefore looks valid but cannot execute its core share lifecycle once it goes live. Existing share holders cannot redeem because `create_redeem_request` cannot burn their shares, and already-approved deposits become irrecoverable because `approve_request` moves user assets into reserve and marks the request `Claimable` before `claim` later fails to mint any shares.

Details `create_vault` checks the share mint only for `supply == 0`. Share-mint extension compatibility is never validated.

```
pub fn handler(ctx: Context<CreateVault>, args: AsyncVaultArgs) -> Result<()> {
    require!(
        ctx.accounts.share_mint.supply == 0,
        AsyncVaultError::ShareMintSupplyShouldBeZero
    );

    validate_asset_mint_extensions_from_acct_info(&ctx.accounts.asset_mint.to_account_info())?;
    ctx.accounts.set_new_authority(ctx.accounts.vault.key())?;
    ...
}
```

Once the vault is initialized, the program assumes that ordinary public share mints and burns will keep working forever. That assumption is encoded in three places:

- Deposit claims mint shares to the user with the standard `mint_to` path.

```
pub fn deposit(&self, seeds: &[&[&[u8]]], shares: u64) -> Result<()> {
    token_interface::mint_to(
        CpiContext::new_with_signer(
            share_token_program.key(),
            MintTo {
                mint: self.share_mint.to_account_info(),
                to: user_share_account.to_account_info(),
                authority: self.vault.to_account_info(),
            },
            seeds,
        ),
        shares,
    )
}
```

- Redemption requests burn shares with the standard public `burn` instruction.

```
pub fn burn_shares(&self, amount: u64) -> Result<()> {
    let cpi_accounts = Burn {
        mint: self.share_mint.to_account_info(),
        from: self.user_share_account.to_account_info(),
        authority: self.user.to_account_info(),
    };
    let cpi_ctx = CpiContext::new(self.share_token_program.key(), cpi_accounts);
    token_interface::burn(cpi_ctx, amount)
}
```

- Canceling or rejecting a pending redemption mints public shares back with the same standard `mint_to` path.

The official Token-2022 processor rejects those regular public mint and burn flows when a mint carries `ConfidentialMintBurn`; they must use the confidential mint/burn extension-specific instruction family instead. Async Vault never does. That creates two concrete failure modes:

- **All live redemptions fail**: any holder trying to create a redeem request hits the unsupported standard `burn` path and cannot exit.
- **Approved deposits can become permanently trapped**: `approve_request` first moves the user's assets from `pending_vault` into `reserve`, computes shares, and marks the request `Claimable`; only the later `claim` call actually mints shares. On a `ConfidentialMintBurn` share mint, that `claim` mint step fails, but the request is no longer pending, so `cancel_request` and `reject_request` cannot unwind it.

The relevant request-state transition in `approve_request` is:

```
ctx.accounts.settle_deposit(seeds, net_deposit)?;
let shares = calculate_shares(nav, decimals, net_deposit)?;
...
request.amount = claimable_amount;
request.request_state = RequestState::Claimable;
```

And the rollback paths only accept `Pending` requests:

```
require!(
    ctx.accounts.request.request_state == RequestState::Pending,
    AsyncVaultError::RequestIsNotPending,
);
```

So once a deposit has been approved under this tainted bootstrap state, the user's assets sit in reserve while the share payout path is permanently incompatible with the chosen share mint.

Impact Cascade

- The vault can be created and initialized around a share mint that is fundamentally incompatible with its own share issuance/burn mechanics.
- Existing share holders lose the ability to redeem because the vault's redemption entrypoint always relies on the standard public burn path.
- Approved deposits can be pushed into `Claimable` state after their assets have already entered reserve, then become unclaimable because the share mint blocks public `mint_to`.
- Those claimable deposits cannot be canceled or rejected back to the user, creating a practically unrecoverable lockup of user value.

Assumptions and Uncertainties

- The share mint uses Token-2022 with the `ConfidentialMintBurn` extension enabled.
- The vault follows the normal share lifecycle shown in the code: public `mint_to` for deposit claims and public `burn` for redemption requests.
- No out-of-band migration or emergency custom tooling is available to rebuild user state after requests have already entered `Claimable`.
- This is a bootstrap compatibility failure; it does not depend on any post-deployment admin abuse.

How will the bug recipient respond? "Confidential mint/burn is a niche Token-2022 feature; integrators should avoid incompatible mints."

That is exactly why the bootstrap ceremony must reject it. The program already performs an asset-side compatibility check and advertises bring-your-own mints as a supported model. If a live vault can be initialized around a share mint for which its own `claim`, `create_redeem_request`, `cancel_request`, and `reject_request` instructions are structurally incompatible, then bootstrap never established a valid share lifecycle in the first place.

Why did tests miss this issue? The creation tests exercise ordinary Token / Token-2022 mints, invalid mint authority, same-mint rejection, asset-side transfer fees, and non-zero public share supply. They never initialize a share mint with `ConfidentialMintBurn`, and the initialize/claim/redeem tests assume standard public mint/burn support on the chosen share mint.

Recommendation Extend share-mint validation to reject Token-2022 extensions that the vault's public share lifecycle cannot support. At minimum, reject `ConfidentialMintBurn` (and any prerequisite extension

combination that disables ordinary public mint/burn) unless the vault is explicitly upgraded to use the confidential mint/burn instruction family throughout claim, redeem, cancel, and reject flows.

References

- programs/async_vault/src/instructions/create_vault.rs#L94-L123
- programs/async_vault/src/instructions/approve_request.rs#L165-L188
- programs/async_vault/src/instructions/approve_request.rs#L231-L235
- programs/async_vault/src/instructions/claim.rs#L79-L103
- programs/async_vault/src/instructions/create_redeem_request.rs#L56-L66
- programs/async_vault/src/instructions/cancel_request.rs#L156-L180
- programs/async_vault/src/instructions/reject_request.rs#L147-L176

4.2.3 VAUL-2 - Reusable request pubkeys let stale signed queue-processing transactions settle a different request instance

✓ **Low**

Status: Fixed

Original severity: Low

Executive Summary Queue requests are created as arbitrary user-supplied accounts, not as PDAs derived from an immutable request nonce or queue ID. The high-value lifecycle instructions then authenticate only the request pubkey plus the *current* on-chain fields stored there. That means the program never binds a signed decision to a specific request instance.

Once a request account is closed, the same keypair can be reused to create a new request at the same pubkey. Any still-valid signed transaction that targeted the old request - most dangerously an authority-signed `approve_request`, but also `reject_request` or a previously signed `set_operator` - will execute against the new request instance instead, because the program re-reads the mutable `Request` and queue-extension data at execution time.

This breaks the queue's core identity guarantee: the authority can review and sign off on request `P` with queue ID `N`, but the chain may actually approve a later request `P` with queue ID `N+1` (or a different amount / NAV context) if the original instance is closed and recreated before the signed transaction expires.

Details Request creation accepts an arbitrary fresh account and does not derive it from any immutable per-request seed:

```
#[account(
    init,
    space = 8 + Request::INIT_SPACE + compute_request_extension_space(&vault.to_account_info()),
    payer = user,
)]
pub request: Account<'info, Request>,
```

The same pattern exists for both deposits and redeems. There is no PDA seed, no monotonic nonce in the instruction args, and no anti-replay field that the later lifecycle instructions must match. [vault/programs/async_vault/src/instructions/create_deposit_request.rs#L41-L46] [vault/programs/async_vault/src/instructions/create_redeem_request.rs#L38-L43]

The authority-facing queue-processing instructions only bind to the request pubkey and the *current* contents at that address:

```
#[account(
    mut,
    has_one = vault @ AsyncVaultError::InvalidRequest,
)]
pub request: Box<Account<'info, Request>>,
```

`approve_request` does not commit the signed decision to an expected queue ID, amount, creation time, or `nav_update_version`; it simply deserializes whichever `Request` currently exists at that pubkey when the transaction lands. [vault/programs/async_vault/src/instructions/approve_request.rs#L33-L37]

Old request instances are explicitly closeable and therefore reusable:

```
#[account(
    mut,
    close = user,
```

```
    constraint = request.owner == user.key() @ AsyncVaultError::UnauthorizedSigner,
    has_one = vault,
)]
pub request: Box<Account<'info, Request>>,
```

```
#[account(
    mut,
    close = owner,
    has_one = owner @ AsyncVaultError::InvalidRequest,
    has_one = vault @ AsyncVaultError::InvalidRequest,
)]
pub request: Box<Account<'info, Request>>,
```

```
#[account(
    mut,
    close = owner,
    constraint = request.request_state == RequestState::Canceled @ crate::error::AsyncVaultError::
    RequestIsNotCanceled,
    has_one = vault,
    has_one = owner,
)]
pub request: Account<'info, Request>,
```

So a queued request can be closed via reject, claim, or tombstone skip, after which the same keypair can immediately be reused for a new request at the same pubkey. `[vault/programs/async_vault/src/instructions/reject_request.rs#L35-L41]` `[vault/programs/async_vault/src/instructions/claim.rs#L36-L42]` `[vault/programs/async_vault/src/extensions/fifo_queues/skip_canceled_queue_request.rs#L17-L24]`

The exploit path is:

- The attacker (or a malicious frontend controlling the ephemeral request keypair) creates request instance `P_old`.
- The authority signs an `approve_request` transaction targeting pubkey `P_old` after reviewing its current queue position and amount.
- Before that signed transaction lands, `P_old` is closed through a normal lifecycle path (for example, the user cancels and the tombstone is skipped, or the original request is otherwise completed/-closed).
- The attacker recreates `P_new` at the exact same pubkey `P`, but with a different queue ID, amount, and approval-time NAV context.
- The still-valid signed approval now executes against `P_new`, because the handler authenticates only the pubkey and current deserialized data.

On Solana, signatures commit to the message's account keys and instruction data, not to the mutable contents of program-owned accounts. Without a per-instance nonce/hash check inside the program, reusing the same request pubkey makes stale signed messages instance-replayable.

`set_operator` has the same structural flaw on the user-signed side - it mutates whichever `Request` currently sits at the reused pubkey - which shows this is a general request-identity problem rather than a single-instruction bug. `[vault/programs/async_vault/src/instructions/set_operator.rs#L5-L20]`

Impact Cascade

- An authority-reviewed approval for queue request `N` can be redirected onto a later request instance with a different queue ID.
- A later redeem request can inherit approval that was only intended for a smaller or earlier request, moving a different asset slice into `pending_vault` than the authority reviewed.
- A later deposit request can inherit approval under a different NAV epoch, turning stale authority intent into unintended share issuance / dilution.
- The queue's FIFO metadata stops being a stable identity boundary once a request pubkey is reusable.
- Any off-chain workflow that signs request-processing transactions ahead of final inclusion inherits this replay surface.

Assumptions and Uncertainties

- The attacker can cause reuse of the same request keypair/pubkey; this is realistic when an attacker-controlled frontend or relayer generates and retains the ephemeral request keypair.
- A signed lifecycle transaction remains valid long enough to land after the old request instance is closed and the new one is created.
- The most severe variant uses a withheld or slow-to-land authority-signed `approve_request`; this is easiest when approvals are routed through relayers / keepers / bundle services rather than landing immediately.
- The program itself does not prevent instance replay once those conditions hold, because it never verifies a per-instance nonce/hash.

How will the bug recipient respond? "This only works if someone reuses the same request keypair and also manages to land an old signed transaction after the original request closes. Our clients generate fresh keys, and Solana transactions expire quickly."

That response describes why the bug is not trivially exploitable, but it does not remove the protocol flaw. The contract-level identity check is still missing: request lifecycle signatures are bound only to a reusable pubkey, not to a unique request instance. Any frontend, relayer, or keeper workflow that reuses request keys or delays signed submission reintroduces the bug immediately, and the chain has no native defense once the stale transaction is still valid.

Why did tests miss this issue? The integration tests always generate a fresh `Keypair::new()` for each request and never model request-pubkey reuse or delayed signed submission, so they only exercise the happy-path assumption that one pubkey corresponds to exactly one request lifecycle. `[vault/integration-tests/src/async_vault/create_deposit_request.rs#L65-L65]`

Recommendation The safest fix is to make request accounts non-reusable from the program's perspective.

Primary fix:

```
#[account(
    init,
    seeds = [b"request", vault.key().as_ref(), user.key().as_ref(), next_request_nonce.to_le_bytes().
    as_ref()],
    bump,
    payer = user,
    space = ...,
)]
pub request: Account<'info, Request>
```

Then store that nonce (or the queue ID) inside `Request`, and require every privileged lifecycle instruction to verify it against an instruction argument or PDA derivation.

Additional hardening:

- Include an immutable `request_instance_nonce` inside `Request` and have `approve_request`, `reject_request`, `claim`, and `set_operator` require the caller to supply the expected nonce.
- Refuse to process request accounts whose creation slot / nonce does not match the authority's signed intent.
- For off-chain workflows, never sign lifecycle transactions until the target request instance is finalized and immediately submitted.

References

- `vault/programs/async_vault/src/instructions/create_deposit_request.rs#L41-L46`
- `vault/programs/async_vault/src/instructions/create_redeem_request.rs#L38-L43`
- `vault/programs/async_vault/src/instructions/approve_request.rs#L33-L37`
- `vault/programs/async_vault/src/instructions/reject_request.rs#L35-L41`
- `vault/programs/async_vault/src/instructions/claim.rs#L36-L42`
- `vault/programs/async_vault/src/extensions/fifo_queues/skip_canceled_queue_request.rs#L17-L24`
- `vault/programs/async_vault/src/instructions/set_operator.rs#L5-L20`
- `vault/integration-tests/src/async_vault/create_deposit_request.rs#L65`